FULL ARCHITECTURE DOCUMENT

MarketPilot

Local market data platform - technical architecture

STREAM	Alpaca -> Kafka -> Spark Streaming -> Gold
CERTIFY	Bronze -> Spark Batch -> Silver -> Gold
SERVE	MariaDB -> Backend API -> Web application

Version 1.0 | 2026-08-29 | Architecture baseline: accepted ADR-001 through ADR-004

ENGINEERING CAPSTONE

01 / TECHNICAL ARCHITECTURE

Executive summary

What the platform does, why it exists, and what this document governs

MarketPilot is a reproducible local data platform that turns market and SEC source data into replayable raw evidence, certified analytical datasets, explainable signals, and a secure browser experience.

Product purpose

Monitor a fixed universe of 10 to 20 US equities plus SPY using one-minute OHLCV bars, technical indicators, explainable signals, historical context, and SEC filing metadata. The product does not execute trades.

Engineering purpose

Demonstrate an end-to-end data platform using Kafka, Spark Streaming and Batch, Airflow, MinIO, MariaDB, Python services, a Backend API, and a Web application under Docker Compose.

Architecture principles

Principle	Meaning in MarketPilot	Primary mechanism
Separate lifecycle ownership	Long-running services and bounded workflows have different supervisors.	Docker Compose for services; Airflow for finite work
Preserve raw evidence	Every modeled result can be traced to immutable source data and replayed.	Kafka offsets, MinIO Bronze, event identifiers
Low latency without sacrificing trust	Live data is usable quickly, while closed sessions are rebuilt and certified.	PROVISIONAL streaming + CERTIFIED batch
Secure the browser boundary	The browser never receives storage or database credentials.	Web -> Backend API -> MariaDB Gold
Design for retries	Duplicate delivery is expected; business state must remain correct.	Unique business keys, checkpoints, idempotent upserts

Document authority

This document summarizes the implemented architecture. The repository source of truth remains **AGENTS.md**, **docs/project-context.md**, **docs/architecture/architecture.md**, **docs/implementation-plan.md**, and accepted ADRs. If implementation and those sources conflict, the conflict must be resolved rather than hidden.

02 / TECHNICAL ARCHITECTURE

Scope, constraints, and quality attributes

The explicit boundaries that keep the local MVP coherent

In scope

One-minute market bars; Alpaca and SEC EDGAR adapters; Kafka transport; immutable Bronze; canonical Silver Parquet; Gold serving models; indicators and signals; data quality; backfill; compaction; annual archive; API and Dashboard.

Out of scope

Trade execution, order routing, portfolio custody, high-frequency latency, multi-broker arbitration, large-scale distributed Kafka, multi-region disaster recovery, and automated MariaDB history purge.

Key constraints

Constraint	Architecture response
Local workstation deployment	Single-broker Kafka KRaft, Spark standalone master/worker, Airflow LocalExecutor, explicit resource discipline.
Small but meaningful workload	Up to approximately 8,190 one-minute events per regular day at 20 equities plus SPY; prioritize correctness over cluster scale.
Market calendar semantics	UTC storage, America/New_York scheduling, and exchange-calendar awareness for holidays and early closes.
External-source fragility	Reconnect and exponential backoff for Alpaca; declared User-Agent, throttling, and accession dedupe for SEC.
At-least-once boundaries	No cross-system exactly-once claim; idempotency is enforced at the business key and publication level.
Cloud migration later	S3-compatible paths and object-storage abstractions keep MinIO replaceable by S3.

Quality attributes

Reliability and recoverability

Durable checkpoints, consumer offsets, named volumes, immutable inputs, idempotent writes, archive manifests, and verified restore drills protect progress and data integrity.

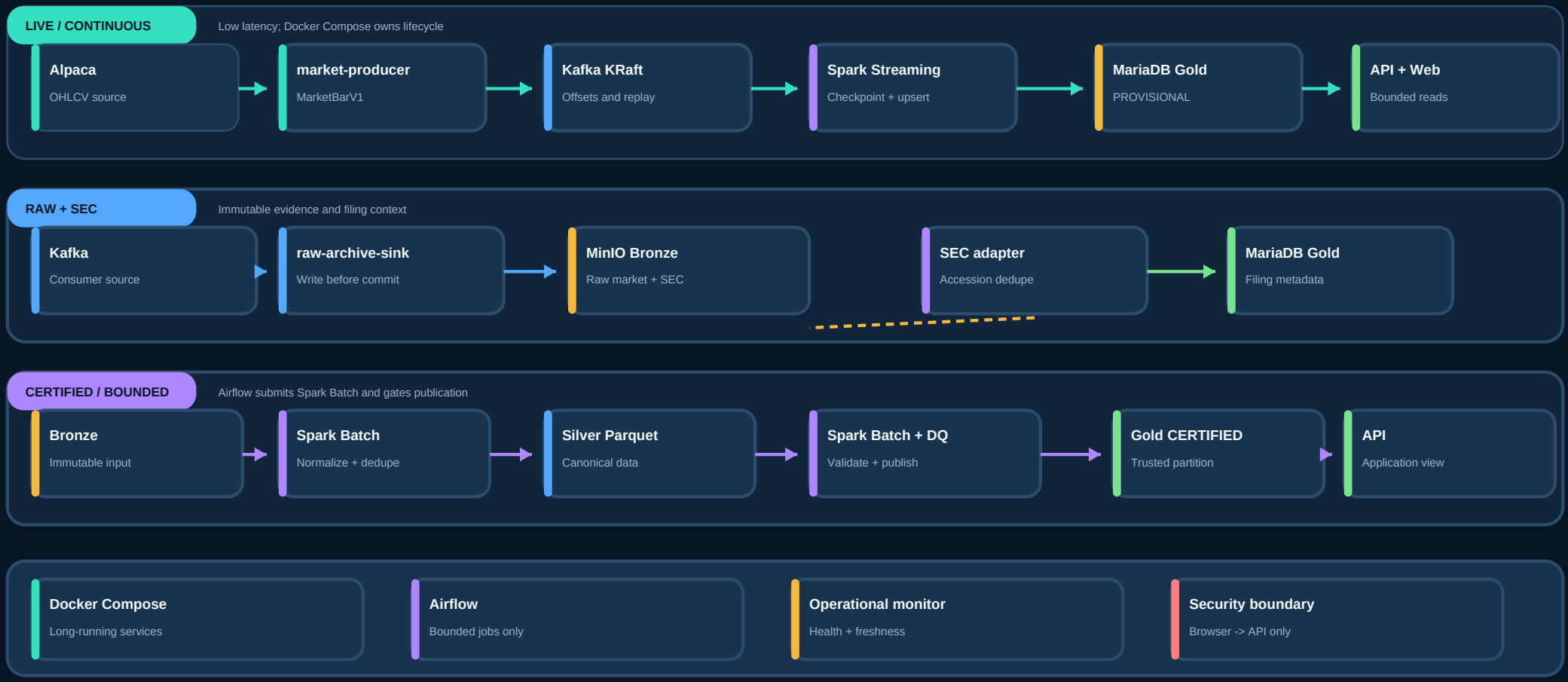
Auditability and explainability

Schema versions, event IDs, source and ingestion timestamps, run/code/data versions, certification state, DQ outcomes, and human-readable Signal explanations retain lineage.

03 / TECHNICAL ARCHITECTURE

System architecture overview

The complete platform organized into continuous, raw, certified, and control planes



The most important boundary: Airflow submits and monitors bounded Spark Batch applications. It never starts, stops, or supervises Spark Structured Streaming or other permanent services.

04 / TECHNICAL ARCHITECTURE

Logical layers and service topology

What each layer owns and how the local runtime is assembled

Layer	Technology	Responsibility	Lifecycle owner
Sources	Alpaca, SEC EDGAR	Market bars and filing source data	External
Ingestion	Python adapters	Connect, validate envelope, publish or archive	Compose or Airflow by workload type
Transport	Kafka KRaft	Buffering, consumer isolation, replay by offset	Docker Compose
Streaming compute	Spark Structured Streaming	Event-time processing, validation, provisional Gold upsert	Docker Compose
Batch compute	Spark Batch	Bronze to Silver, Silver to Gold, analytics, backfill	Airflow submission
Orchestration	Airflow LocalExecutor	Schedules, dependencies, retries, timeouts, pools	Docker Compose hosts Airflow
Object storage	MinIO; future S3	Bronze, Silver, manifests, archives	Docker Compose
Serving storage	MariaDB	Application-ready Gold data	Docker Compose
Application	Backend API, Nginx Web App	Bounded secure access and user experience	Docker Compose
Operations	Operational monitor + logs	Dependency, freshness, alert state transitions	Docker Compose

Runtime service groups

Data plane

kafka, market-producer, raw-archive-sink, spark-master, spark-worker, spark-streaming, minio, mariadb, backend-api, web-app, operational-monitor.

Orchestration plane

airflow-db, airflow-scheduler, airflow-api-server, airflow-dag-processor, plus bounded submission clients for Spark Batch, SEC polling, compaction, archive, and restore workflows.

Persistent state

Named volumes protect Kafka logs, MinIO objects, MariaDB data, Airflow metadata and logs, and Spark checkpoints. Object storage retains immutable and analytical data independently of the serving database.

05 / TECHNICAL ARCHITECTURE

Data flow and publication semantics

Four paths serve different latency, recovery, and trust requirements

Path	Route	Execution	Result
Live market	Alpaca -> market-producer -> Kafka -> Spark Structured Streaming -> MariaDB Gold	Continuous	PROVISIONAL low-latency bars; idempotent upsert
Raw archive	Kafka -> raw-archive-sink -> MinIO Bronze	Continuous	Immutable event evidence; offset commit after durable write
Certified market	Bronze -> Spark Batch -> Silver -> DQ -> Spark Batch -> Gold	Bounded	Authoritative closed-session rebuild and certification
SEC filings	SEC EDGAR -> SEC adapter -> MinIO Bronze + MariaDB Gold	Bounded polling	Raw JSON in Bronze; accession-keyed metadata in Gold

Provisional versus certified

Streaming optimizes for freshness. Batch optimizes for completeness and trust. A failed batch never hides the last known certified partition, and no publication watermark advances after a failed quality gate.

Why the live path exists

Users should not wait until market close to see recent bars. Spark Streaming consumes micro-batches, applies event-time semantics, and publishes PROVISIONAL rows that can be safely overwritten by the same business key.

Why the certified path exists

Late events, source corrections, and complete-session validation require reconstruction from immutable Bronze. The batch pipeline rebuilds a partition, applies blocking checks, and publishes CERTIFIED state atomically.

Browser serving flow

The browser calls relative **/api/** routes through Nginx. The Backend API applies validation, bounded date ranges, pagination, response models, and a narrowly scoped read-only MariaDB identity. Internal Bronze URIs and credentials are not exposed.

06 / TECHNICAL ARCHITECTURE

Events, schemas, lineage, and time

The metadata required to understand every record and replay every path

MarketBarV1 event contract

Field group	Required fields	Purpose
Identity	event_id, event_type, schema_version	Stable event identity and explicit compatibility contract
Source time	source, source_event_time	When and where the market observation originated
Ingestion	ingested_at_utc, correlation_id	Arrival timing and cross-service traceability
Business payload	symbol, interval, open, high, low, close, volume	Typed one-minute OHLCV observation
Kafka provenance	topic, partition, offset	Recoverable transport position retained in archive path/metadata

Business uniqueness

Market bars: symbol + event UTC timestamp + interval. SEC filings: accession number. Indicators: symbol + timestamp + indicator code + version. Signals: symbol + signal timestamp + model version.

Published lineage

Every published dataset carries source reference, run ID, code version, data version, schema or model version, and certification state. DQ results and watermarks record the decision to publish.

Time model

Concern	Rule
Storage	All event and processing timestamps are stored in UTC.
Schedules	Market and Airflow definitions use America/New_York, not a fixed UTC offset.
Trading sessions	An exchange calendar handles holidays, regular sessions, and early closes.
Late data	Live state remains provisional; batch reconstruction is authoritative for closed sessions.
Freshness	Operational monitoring compares source, ingestion, and Gold timing without fabricating missing bars.

07 / TECHNICAL ARCHITECTURE

Storage architecture and Gold model

Each storage engine is used for the workload it serves best

Zone	Technology	Contents	Write model
Bronze	MinIO / future S3	Immutable market events, SEC raw JSON, recoverable metadata	Append-oriented; no silent mutation
Silver	MinIO Parquet	Canonical typed schemas, normalized timestamps, dedupe and validation flags	Partitioned batch output; compacted safely
Gold	MariaDB	Bars, indicators, signals, SEC metadata, watermarks, DQ, manifests	Idempotent upsert or atomic partition replacement
Archive	MinIO Parquet	Closed annual exports, inventory, SHA-256 digests, schema/code version	Immutable versioned objects

Gold serving model

Table	Business key	Responsibility
dim_symbol	symbol	Asset metadata and stable symbol identifier
fact_market_bar_1m	symbol ID + event UTC timestamp	One-minute OHLCV with provisional/certified state
fact_indicator_1m	symbol + timestamp + indicator + version	SMA, RSI, realized volatility, volume ratio
fact_signal	symbol + timestamp + model version	Sparse explainable observations
fact_sec_filing	accession number	Filing metadata and Bronze reference
etl_watermark	pipeline + partition	Progress and publication state
data_quality_result	run + check + partition	Measured quality evidence
archive_manifest	dataset + year + version	Verified archive inventory and restore metadata

Raw payloads and long-term analytical history are never stored only in MariaDB. The serving database remains optimized for application queries while object storage preserves replay and archive capabilities.

08 / TECHNICAL ARCHITECTURE

Execution and orchestration model

The platform distinguishes permanent processes from work that reaches a terminal state

Component	Started by	Completion model	Restart / retry owner
Kafka	Docker Compose	Long-running	Docker restart policy
Market producer	Docker Compose	Long-running	Docker + source reconnect logic
Raw archive sink	Docker Compose	Long-running	Docker + recoverable Kafka offsets
Spark Structured Streaming	Docker Compose	Long-running	Docker + durable Spark checkpoint
Spark Batch	Airflow via SparkSubmitOperator	Finite application	Airflow retries and timeout
SEC polling	Airflow	Finite request batch	Airflow retries and pool
Compaction / archive / restore	Airflow or operator	Finite controlled job	Workflow retry and manifest state
MariaDB / MinIO / API / Web	Docker Compose	Long-running	Docker health and restart policy

DAG catalogue

DAG	Schedule / trigger	Bounded responsibility
sec_polling_dag	Every 15 minutes in configured weekday window	Discover, download, deduplicate, archive, watermark
daily_market_close_dag	16:30 America/New_York on trading days	Completeness, Bronze to Silver, DQ, Gold, analytics, publish
weekly_compaction_dag	Saturday 06:00 New York	Compact Silver, validate equivalence, publish manifest
annual_archive_dag	January 10 02:00 New York	Export prior year, verify inventory and checksums
backfill_replay_dag	Manual validated parameters	Replay selected dates and symbols without catchup ambiguity

Concurrency controls: **catchup=False** for routine polling and daily DAGs; **max_active_runs=1** for partition-mutating DAGs; dedicated SEC and Spark pools; explicit retries, exponential backoff, timeouts, and parameter validation.

09 / TECHNICAL ARCHITECTURE

Data quality, idempotency, and failure semantics

Correctness is protected at every boundary instead of assumed end to end

Blocking quality dimensions

Dimension	Example control	Publication effect
Freshness	Latest event and Gold watermark against expected session	Alert or block stale certified publication
Completeness	Expected bars by symbol and exchange session	Block incomplete closed partitions
Duplicates	Business-key counts before and after dedupe	Block unresolved duplicate keys
Nulls and schema	Required fields, explicit schema version, typed values	Quarantine malformed records
OHLC consistency	high >= open/close/low and low <= open/close/high	Block invalid market bars
Analytics ranges	RSI and Signal strength domain checks	Block invalid analytical publication

Failure behavior

Failure	Expected outcome
Producer disconnect	Reconnect with bounded exponential backoff; never fabricate bars.
Kafka unavailable	Producer retries; consumer positions remain recoverable.
Streaming crash	Compose restarts; Spark resumes from checkpoint.
Malformed event	Quarantine or DLQ with reason and source metadata.
Batch or DQ failure	Airflow retries; no publication watermark advances.
MariaDB write ambiguity	Retry only through the idempotent business-key path.
Archive verification failure	Manifest remains unverified; purge is forbidden.

MarketPilot does not claim exactly-once delivery across Kafka, Spark, MinIO, and MariaDB. It guarantees business correctness through replayable input, durable progress, deterministic transformations, unique keys, and idempotent publication.

10 / TECHNICAL ARCHITECTURE

Security and network boundaries

Least privilege and clear trust zones limit the effect of a compromised component

Browser trust boundary

The browser communicates only with the Web App and Backend API. It never connects directly to MariaDB, MinIO, Kafka, Spark, or Airflow and never receives infrastructure credentials or internal object URIs.

Identity separation

Ingestion and publication identities may write only their required targets. The application identity has narrow SELECT privileges. Airflow metadata PostgreSQL remains separate from business MariaDB.

Logical network zones

Network	Primary members	Purpose
ingestion-net	Producer, Kafka, raw sink, SEC adapter	Source connectivity and event ingress
processing-net	Kafka, Spark master/worker, streaming and batch	Compute and transport connectivity
storage-net	MinIO, MariaDB, Spark, archive, API	Controlled data persistence access
orchestration-net	Airflow components, metadata DB, Spark master, SEC adapter	Bounded job submission and monitoring
serving-net	Backend API, Web App	User-facing application boundary

Security controls

Control	Implementation intent
Secret handling	Real keys and passwords remain in local environment configuration and are never committed.
Port exposure	Only developer-required host ports are published; internal service traffic uses Docker networks.
Input validation	API symbols, dates, page size, backfill ranges, and archive periods are bounded and validated.
No Docker socket in Airflow	Avoids host-level control unless a separately documented risk decision approves it.
No automatic purge	Archive export does not delete MariaDB history; retention requires a new ADR and verified restore.

11 / TECHNICAL ARCHITECTURE

Observability, operations, and recovery

The platform exposes enough evidence to diagnose freshness, progress, and failure

Signals and metrics

Area	Minimum evidence
Ingestion	Producer connection state, reconnect count, Kafka publish failures
Transport	Consumer lag, topic/partition/offset progress, DLQ volume
Processing	Streaming micro-batch duration, checkpoint progress, batch duration and terminal state
Data quality	Expected versus actual bars, duplicates, rejected records, provisional/certified reconciliation
Serving	Gold freshness, API health, MariaDB connection and query latency
Object storage	Object counts, small-file growth, compaction outcomes, manifest verification

Operational monitor

A long-running, read-only Compose service probes dependencies and freshness. It emits structured JSON logs and deduplicated state transitions instead of repeated alerts. A generic webhook is optional.

Runbooks

Operational procedures cover streaming restart, checkpoint safety, Kafka/MinIO/MariaDB boundaries, archive and compaction failure, backup, verified restore, and presentation fallback evidence.

Archive and restore

Annual archive exports eligible closed periods into immutable, versioned Parquet objects. A deterministic inventory records SHA-256 digests, row counts, time bounds, run ID, code version, schema version, and closed-period state. Restore first verifies the complete inventory and writes into an isolated restore schema. MariaDB history is not automatically purged.

Capacity baseline

The local stack targets an 8-core workstation, 16 GB RAM minimum for comfortable concurrent operation, and approximately 100 GB SSD capacity. Spark must not consume all host memory; Kafka, Airflow, MariaDB, MinIO, the API, Docker, and the operating system need reserved capacity.

12 / TECHNICAL ARCHITECTURE

Architecture decisions and tradeoffs

Accepted decisions make the system reviewable and prevent accidental boundary erosion

Decision	Accepted outcome	Tradeoff
ADR-001 Storage strategy	MariaDB serves Gold; MinIO stores Bronze, Silver, and archives.	Two storage systems require reconciliation and restore discipline.
ADR-002 Airflow boundary	Airflow owns bounded work only; Compose owns permanent services.	Service health monitoring is separate from DAG state.
ADR-003 Streaming lifecycle	Streaming starts automatically, checkpoints progress, and uses idempotent upserts.	Checkpoint state is critical and exactly-once is not assumed.
ADR-004 Publication state	Streaming writes PROVISIONAL; batch publishes CERTIFIED after DQ.	Consumers and metrics must understand two trust states.

Deliberately rejected or deferred

Rejected

All data in MariaDB; Spark Streaming as an Airflow task; browser access to storage; silent record dropping; automatic purge after archive; unbounded API scans.

Deferred

S3 migration, larger Kafka topology, custom Airflow timetable, advanced streaming indicators, Celery/Redis, centralized log indexing, multi-region recovery, and automated retention policy.

Future evolution

Evolution	Protected invariant
MinIO -> S3	Bronze remains immutable; Silver stays Parquet; manifests and lineage remain verifiable.
Single Kafka broker -> cluster	Topic contracts and business idempotency remain unchanged.
Local Spark -> managed compute	Streaming and batch remain separate execution models.
Local Web -> hosted application	Browser still communicates only with a bounded Backend API.
Expanded analytics	Every model remains versioned, explainable, quality-gated, and lineage-aware.

13 / TECHNICAL ARCHITECTURE

Reviewer checklist and references

A concise guide for validating the architecture against implementation evidence

Reviewer checklist

Question	Evidence to inspect
Are long-running and bounded workloads separated?	docker-compose.yml, execution-model.md, Airflow DAGs, ADR-002
Can raw data be replayed?	Kafka offsets, raw-archive-sink, MinIO Bronze layout, event IDs
Is closed-session publication trustworthy?	Bronze to Silver to Gold DAG, DQ results, certification watermark
Are retries safe?	Business keys, MariaDB upserts, Spark checkpoint, duplicate integration tests
Is the browser isolated from data infrastructure?	Nginx relative /api/ proxy, Backend API models, read-only DB grants
Can archived data be restored?	Archive manifest, SHA-256 inventory, isolated restore drill, Phase 8 evidence
Are analytics explainable?	Versioned Indicator/Signal tables, explanations, Phase 9 quality evidence

Repository references

Reference	Role
AGENTS.md	Mandatory architecture, development standards, quality gates, and working agreement
docs/project-context.md	Purpose, scope, data sources, success criteria, and security boundaries
docs/architecture/architecture.md	Logical layers, service topology, storage, DAGs, API, observability
docs/architecture/execution-model.md	Lifecycle ownership, automatic streaming, Airflow/Spark interaction, failures
docs/decisions/ADR-001..004	Accepted storage, orchestration, streaming, and certification decisions
docs/implementation-plan.md	Phase-by-phase delivery and acceptance criteria
docs/phase3-verification.md through docs/phase10-verification.md	Dated implementation and runtime evidence
docs/runbooks/	Operational, failure, archive, backup, and restore procedures

Architecture status: implemented local engineering capstone. This document describes the current local MVP and the accepted boundaries that future changes must preserve.